```
"""Complex elements with many inputs and/or outputs"""

from __future__ import division

import utils, basic
from connect import MultiQueue, MultiInject


class Router(MultiInject):

    """Forwards each incoming car into one of several destinations.

    Which one is served is determined by calling self.select(car) that should
    be implemented in a subclass and return the name of the queue to serve.

    """

    def __init__(self):
        super(Router, self).__init__()
        MultiInject.__init__(self)
    def append(self, car):
        self.outputs[self.select(car)].target.append(car)


class RandomRouter(Router):

    """Selects randomly with fixed probabilities (per source road).

    `turn_probs` should map from car.input_name (set by last multi_server) to
    a map defining probability distribution over output names to turn probabilies.

    """

    def __init__(self, turn_probs = None):
        super(RandomRouter, self).__init__()
        self.probs = turn_probs
    def select(self, car):
        probs = ((self.probs and self.probs[car.input_name])
                 or dict([(name, 1) for name in self.outputs]))
        return utils.rand_select(probs)


class SwitchedServer(MultiQueue, basic.Server):

    """Serves one of several queues each tick.

    Which one is served is determined by calling a chain of "filter" functions
    until at most one name remain.  Each function is passed the SwitchedServer
    object and a list of queue port names and should return new (possibly
    empty) set of names (usually a subset of it).  Specifying a tuple of
    function, args and possibly keyword args allows to give it additional
    args.  At the end of the chain, no more than one name should remain.

    The served queue name (if any) is stored in `self.last_name`.  When the
    choice changes, serving is suspended by `yellow` cycles.

    """

    def __init__(self, filter_chain, yellow=0):
        """Set the chain to be `filter_chain`."""
        super(SwitchedServer, self).__init__()
        self.filter_chain = filter_chain
        self.yellow = yellow
        self.yellow_countdown = 0
        self.last_name = None

    def tick(self):
        if self.yellow_countdown:
            self.yellow_countdown -= 1
            if self.yellow_countdown == 0:
```

```
                names = [self.resume_name]
            else:
                names = []
        else:
            names = self.inputs.keys()
            def myapply(func, args=(), kwargs={}):
                return func(self, names, *args, **kwargs)
            for filter in self.filter_chain:
                if isinstance(filter, tuple):
                    names = myapply(*filter)
                else:
                    names = myapply(filter)
                if len(names) <= 1:
                    break
        if names:
            name = names[0]
            if self.last_name != name:
                self.resume_name = name
                self.yellow_countdown = self.yellow
            self.last_name = name
            if self.yellow_countdown:
                return
            queue = self.inputs[name]
            if len(queue) != 0:
                car = queue.pop(0)
                car.input_name = name
                self.target.append(car)


def nonempty_filter(serv, names):
    """Filter out empty queues."""
    return [name for name in names if len(serv.inputs[name]) != 0]


def rand_filter(serv, names, probs = None):
    """Select randomly with fixed probabilities."""
    if not probs:                       # Equal prob. to each
        probs = dict([(name, 1) for name in names])
    name_num = utils.rand_select([probs[name] for name in names])
    return [names[name_num]]


def attempt_hold_filter(serv, names):
    """Choose `serv.last_name` if it is in names.

    Also, if suceesful, prolong `hold_filter`'s countdown (if one is used).

    """
    last = serv.last_name
    if last in names:
        serv.hold_countdown = 0
        return [last]
    else:
        return names


def hold_filter(serv, names, cycles):
    """Allow changes exactly once in `cycles` cycles.

    Once in N cycles it will let names through and the following filters will
    actually decide.  Rest of the time it will return the last name, ignoring
    the ones it got.

    Put this before everything.  You can only use one such filter in a server.

    """
    if getattr(serv, 'hold_countdown', 0) > 0 and serv.last_name is not None:
        serv.hold_countdown -= 1
        return [serv.last_name]
    else:
```

```
        serv.hold_countdown = cycles - 1
        return names


def lru_filter(serv, names):
    """Select one in least-recently-used manner.

    For stable names list, results in round-robin behaviour.
    You can only use one such filter in a server.

    """
    if not hasattr(serv, 'lru_order'):
        serv.lru_order = []
    lru = serv.lru_order
    # Prefer name closest to list start
    best_index = len(lru)
    for name in names:
        try:
            index = lru.index(name)
        except ValueError:              # not in list
            lru.append(name)
            return [name]
        if index < best_index:
            best_index = index
            best_name = name
    del lru[best_index]
    lru.append(best_name)
    return [best_name]


def longest_queue_filter(serv, names):
    """Select the longest queue(s)"""
    best = -1
    for name in names:
        l = len(serv.inputs[name])
        if l > best:
            best = l
            best_names = [name]
        elif l == best:
            best_names.append(name)
    return best_names


def last_emptying_filter(serv, names, window = 10):
    """Select the queue predicted to become empty last (len/(1-load)).

    Requires some LoadStatQueue to be used.

    """
    best = -1
    for name in names:
        queue = serv.inputs[name]
        l = len(queue) / (1.001 - queue.estimate_load()) # avoid zero division
        if l > best:
            best = l
            best_names = [name]
        elif l == best:
            best_names.append(name)
    return best_names


def inorder_filter(serv, names):
    """Select the car(s) waiting the most.

    Requires some StatQueue to be used.

    """
    best = -1
    best_names = []
    for name in names:
```

```
        queue = serv.inputs[name]
        if not queue:
            continue
        l = queue[0].waited_here
        if l > best:
            best = l
            best_names = [name]
        elif l == best:
            best_names.append(name)
    return best_names


# I hope this is the right order...
#
# At the "end of the day" a car is either somewhere or in a queue.
# Only then it's considered waiting.
phases = [basic.Source, basic.Road, basic.Server, basic.Queue, basic.Car, basic.Drain]


# For testing:

def optimal_blind_probs(source_probs):
    def sum(seq):
        return reduce(lambda a,b:a+b, seq)
    probs = source_probs[:]
    left = 1 - sum(probs)
    bonus = [p*(1-p) for p in probs]
    bonus = [b / sum(bonus) * left for b in bonus]
    return map(lambda a,b:a+b, probs, bonus)
```